TEACH-ME SESSION · 60 MINUTES

Running jobs with Airflow

Trigger our scripts from a browser, with a form for the inputs and a record of every run.

[Presenter name] · [Team]

[Date]

THE HOUR

Agenda

Minutes	Topic	You leave knowing
0–5	Why Airflow	Today’s process vs. with Airflow
5–15	Key ideas	Tasks, DAGs, the job file
15–20	Our setup	What runs on the server
20–35	Using the web page	Find, trigger, watch, read logs, fix
35–45	Writing your own job	The template, common problems
45–55	Hands-on	Running it yourself
55–60	Questions	

Appendix: the example jobs, extra diagrams, glossary, cheat sheet and references.

WHY AIRFLOW

Today, every run goes through one person

Today

You need a report
e.g. East sales over 100

→

Email the person with server access
inputs typed in the message

⚠ inputs live in an email

→

Wait until they're free
could be hours or days

⚠ only 1–2 people can run it

→

They run the script by hand
logging in, typing your inputs

⚠ easy to mistype an input

→

Results emailed back
if nothing went wrong

⚠ no record of what ran

With Airflow

Open the Airflow page
from any machine on the network

→

Fill in the form, click Trigger
inputs checked before it runs

→

The server runs the steps
in order, retrying if needed

→

Results + a record of the run
who, when, which inputs, logs

✓ no waiting on one person

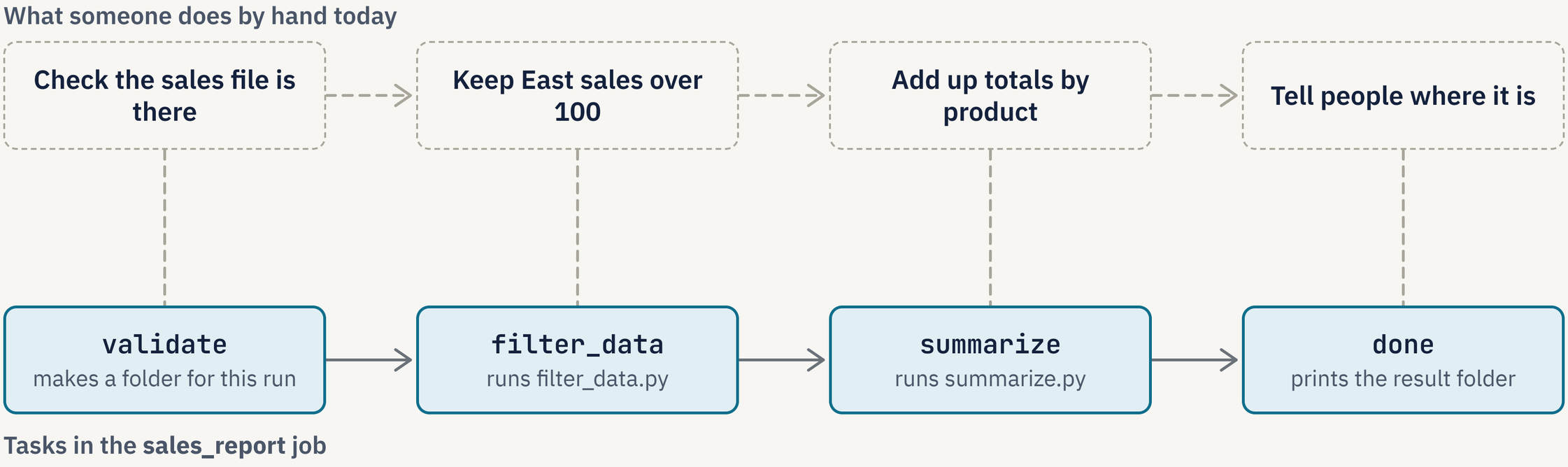
✓ anyone with access can run it

✓ same steps every time

✓ every run is recorded

FROM MANUAL STEPS TO A WORKFLOW

Each manual step becomes a task



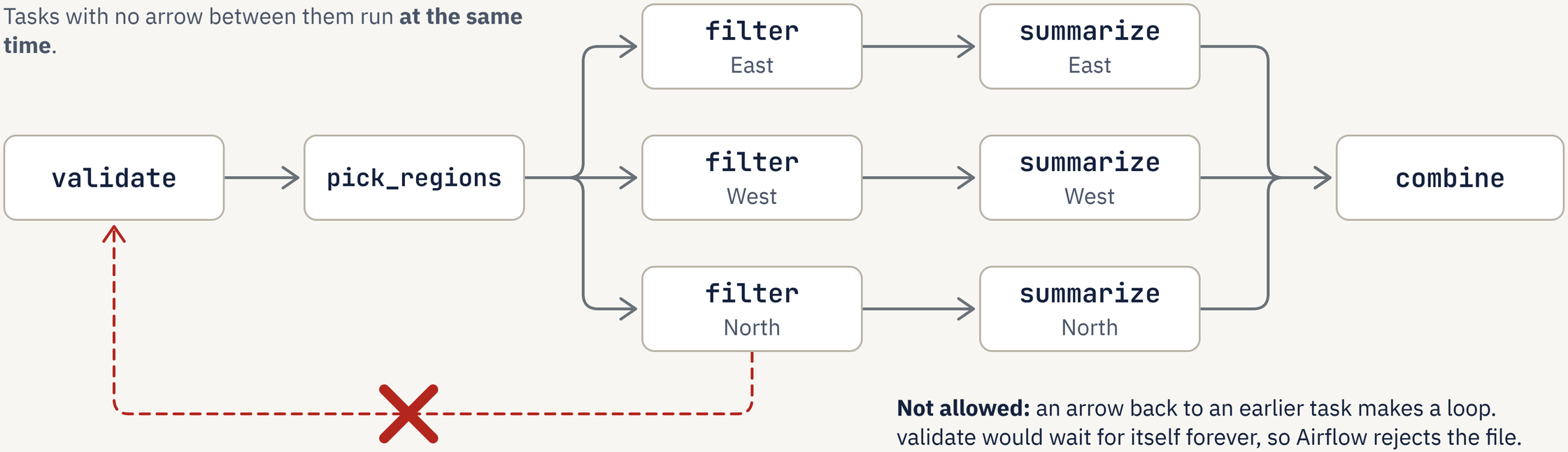
Same steps, same order. The difference: anyone can start it, with their own inputs, and every run is recorded.

WHAT A DAG IS

A job is a graph of tasks with no loops

Directed: arrows point one way. **Acyclic**: no path leads back. **Graph**: tasks joined by dependencies.

Tasks with no arrow between them run **at the same time**.



Not allowed: an arrow back to an earlier task makes a loop. validate would wait for itself forever, so Airflow rejects the file.

ANATOMY OF A DAG

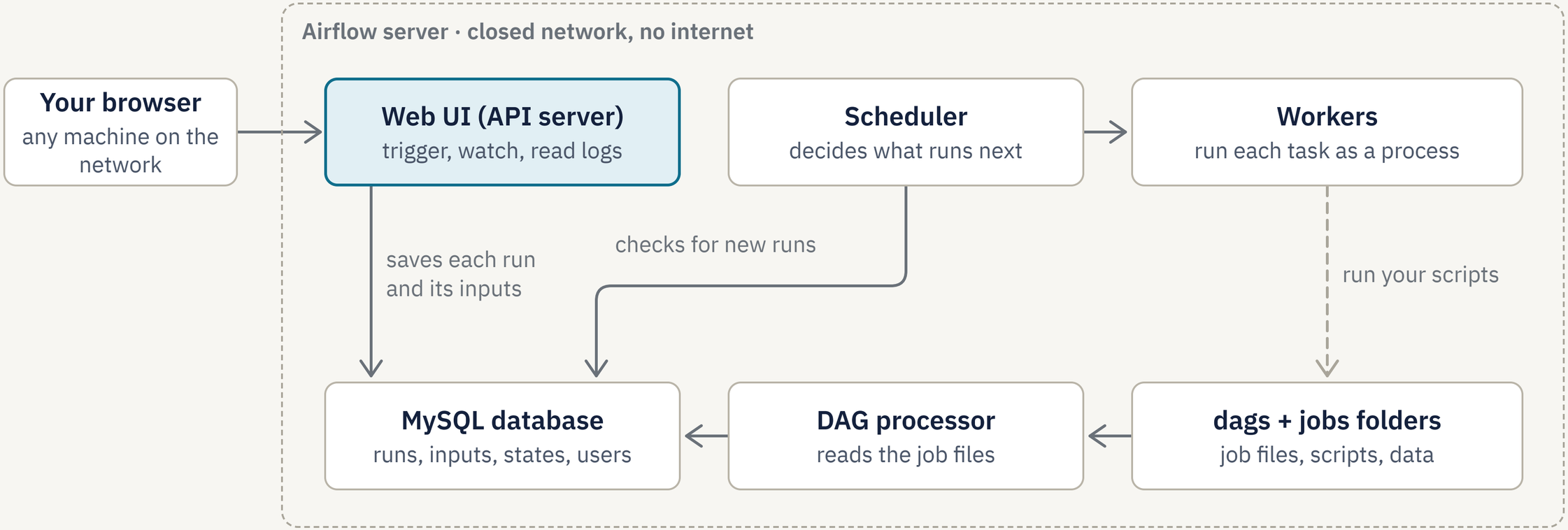
A job is one short Python file

```
@dag(
    dag_id="sales_report",
    schedule=None,
    params={...},
)
def sales_report():
    filter_data = BashOperator(...)
    summarize = BashOperator(...)
    filter_data >> summarize
```

1. **dag_id** is the name you see in the UI
2. **schedule=None** means it runs only when triggered
3. **params** are the form fields you fill in
4. **Each operator** is a task
5. **>>** sets the order

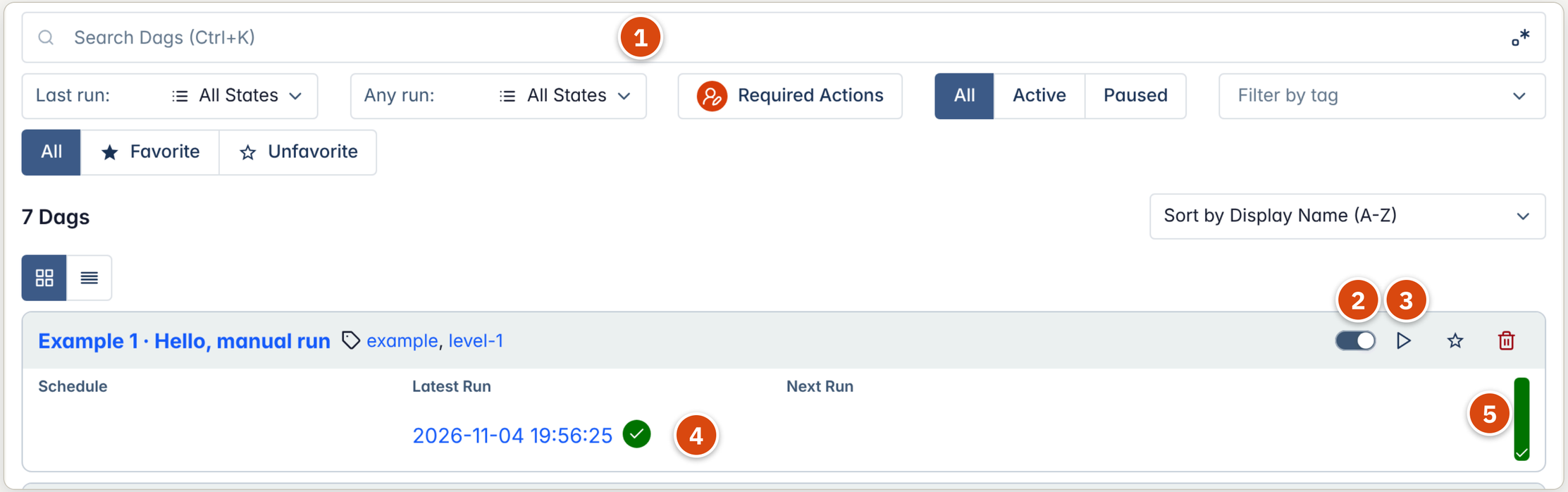
OUR SETUP

What runs on the Airflow server



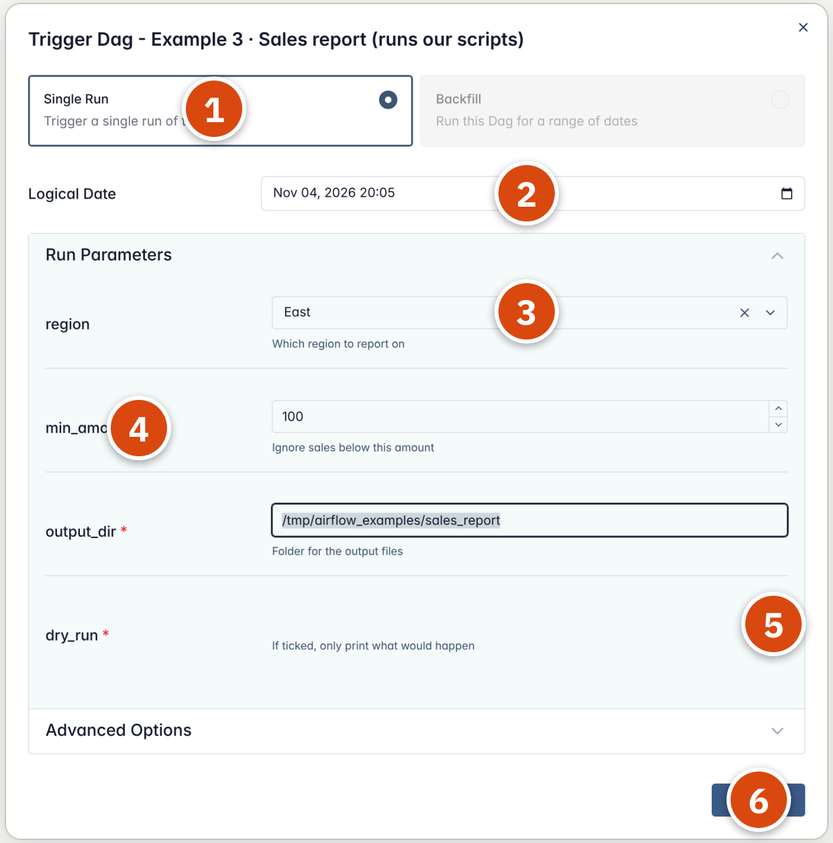
Airflow 2 calls the web UI the *webserver*, and its scheduler reads the job files itself.

Find your job on the Dags page



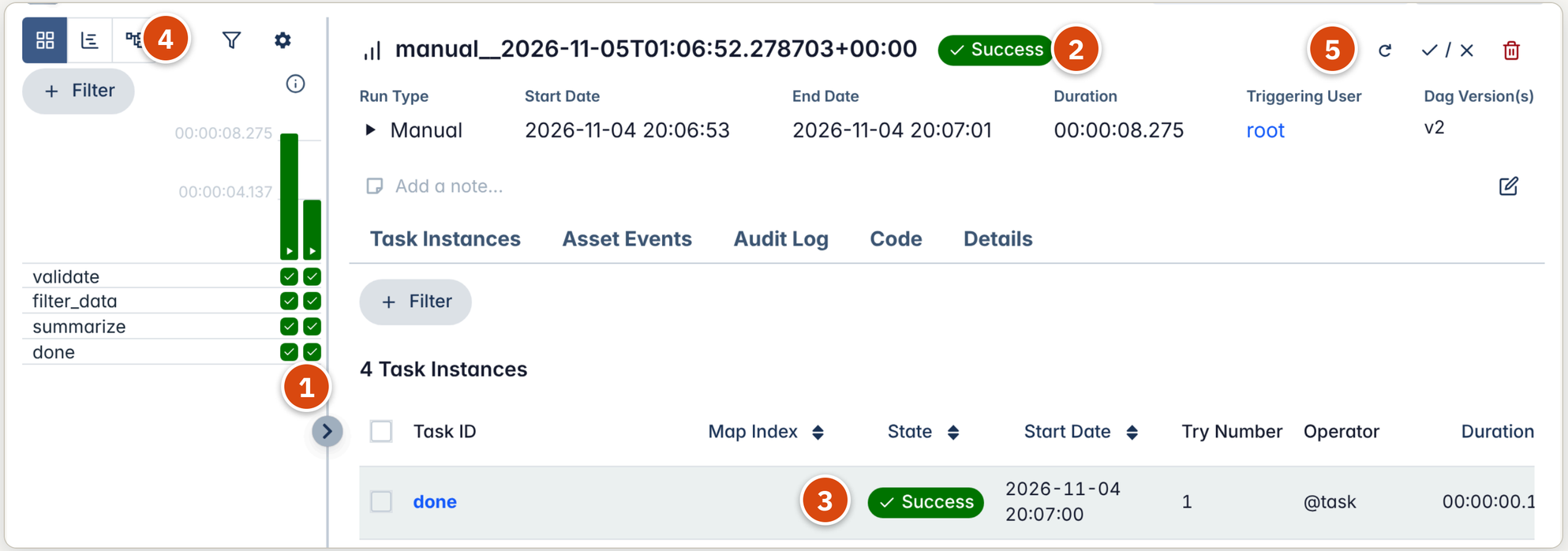
1. **Search** by name or tag
2. **Toggle** on, or it won't run
3. ▶ opens the form
4. **Green tick**: last run worked
5. **Bars**: recent runs, click one

Fill in the form, then click Trigger



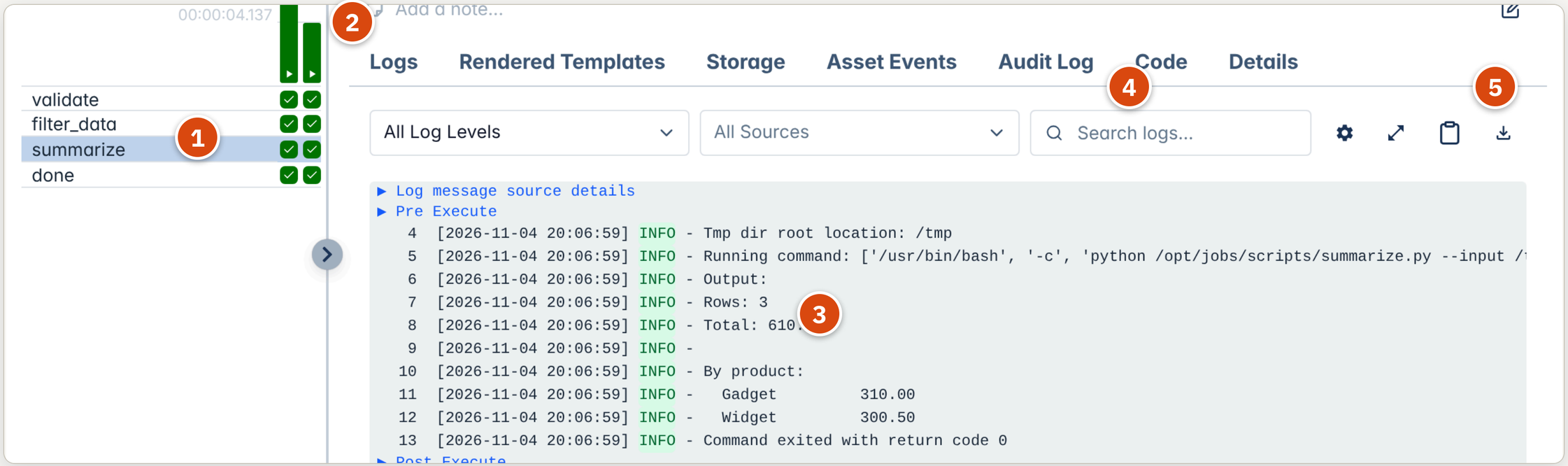
1. **Single Run**: leave it selected.
2. **Logical Date**: a label for the run. Leave the default.
3. Each field comes from the job's **params**. Dropdowns only offer valid choices.
4. ***** means required. Bad values are refused before anything runs.
5. **Toggles** are yes/no options, like a dry run.
6. **Trigger** starts the run and opens its page.

Watch the run: one square per task



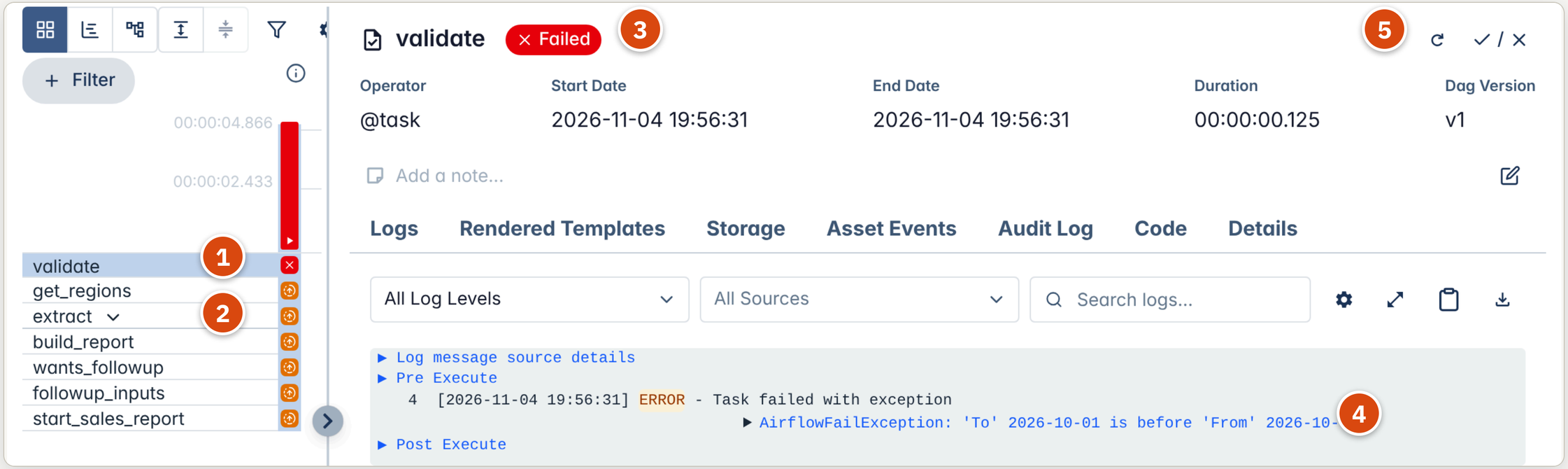
1. **Grid**: column = run, row = task
2. The **run's result**
3. Each **task's state**
4. **Graph**: boxes and arrows
5. **Clear**: run it again

Read what a task printed



1. Click the **task** in the grid
2. **Logs** tab opens first
3. Your script's **output**
4. **Search** the log
5. **Download** the log

When a task goes red



1. **Red**: this task failed
2. **Orange**: never ran
3. The task's **status**
4. The **error message**
5. Fixed it? ↻ **Clear**

WRITE YOUR OWN

Wrap an existing script in four edits

```
@dag(dag_id="my_job", schedule=None,
     start_date=datetime(2024, 1, 1),
     params={
       "input_file": Param("/data/in.csv"),
       "limit": Param(100, type="integer"),
     })
def my_job():
    BashOperator(
        task_id="run_script",
        bash_command="python /opt/jobs/scripts/my_script.py "
          "--input {{ params.input_file }} "
          "--limit {{ params.limit }}",
    )
my_job()
```

1. Give it a unique **dag_id**

2. List your inputs in **params**

3. Point **bash_command** at your script, by full path

4. Pass inputs with **{{ params.name }}**

Save it in the dags folder. It appears in the UI within a couple of minutes.

TROUBLESHOOTING

Five problems you will probably hit

You see	Fix
Job missing from the list	Run airflow dags list-import-errors
Toggle is off	New jobs start paused: switch it on
Trigger button greyed out	You have Viewer; ask for User
“File not found” in a task	Use full paths; check the file is on the server
Run stuck in “queued”	Scheduler is down: tell the admin

When a task fails, the last lines of its Logs tab almost always name the cause.

HANDS-ON · 10 MINUTES

Your turn: run the sales report four ways

1. Dry run

Tick dry_run. Read the filter_data log: how many rows match?

2. West over 100

Open the summarize log and note the total.

3. All regions

Compare the per-product totals with your West run.

Stretch: your script

Copy the template, point it at one of your scripts, run airflow dags test.

Questions?

Airflow page: http://<server>:8080

Accounts: [admin name]

Handout: setup, template, troubleshooting and exercises

Code and slides: github.com/lllatsyrC/airflow-demo

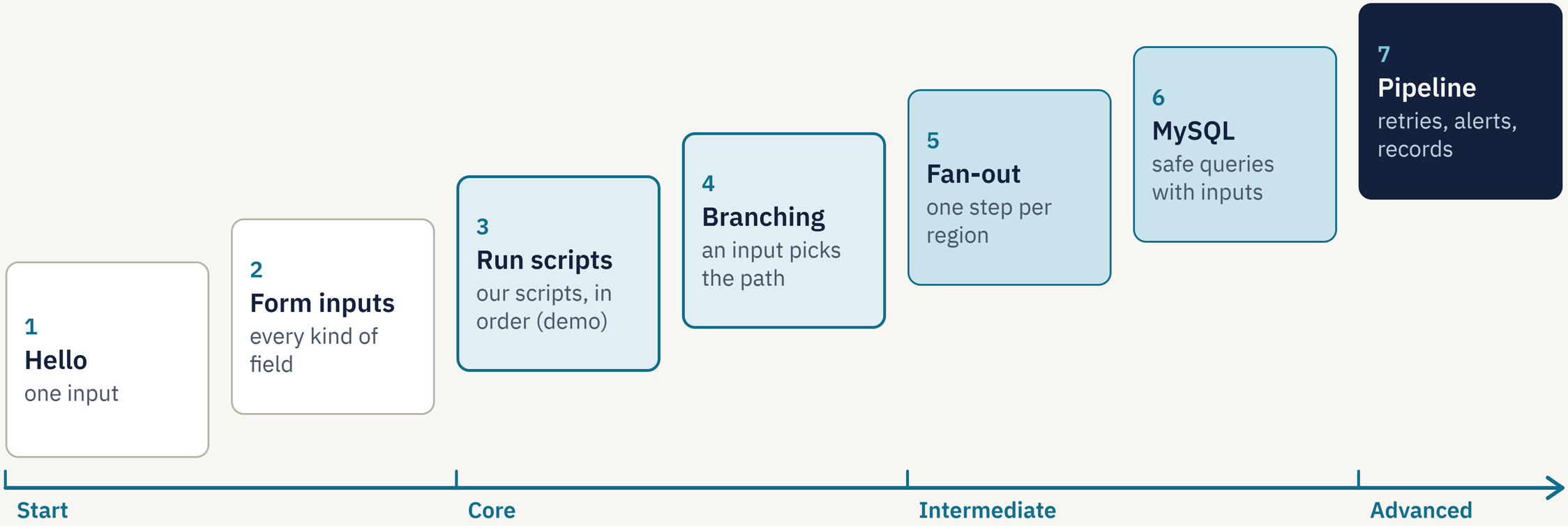
APPENDIX

Example jobs and reference

The seven example jobs and how they build on each other · how **>>** draws arrows · what happens after Trigger · glossary · where files go · command cheat sheet · references

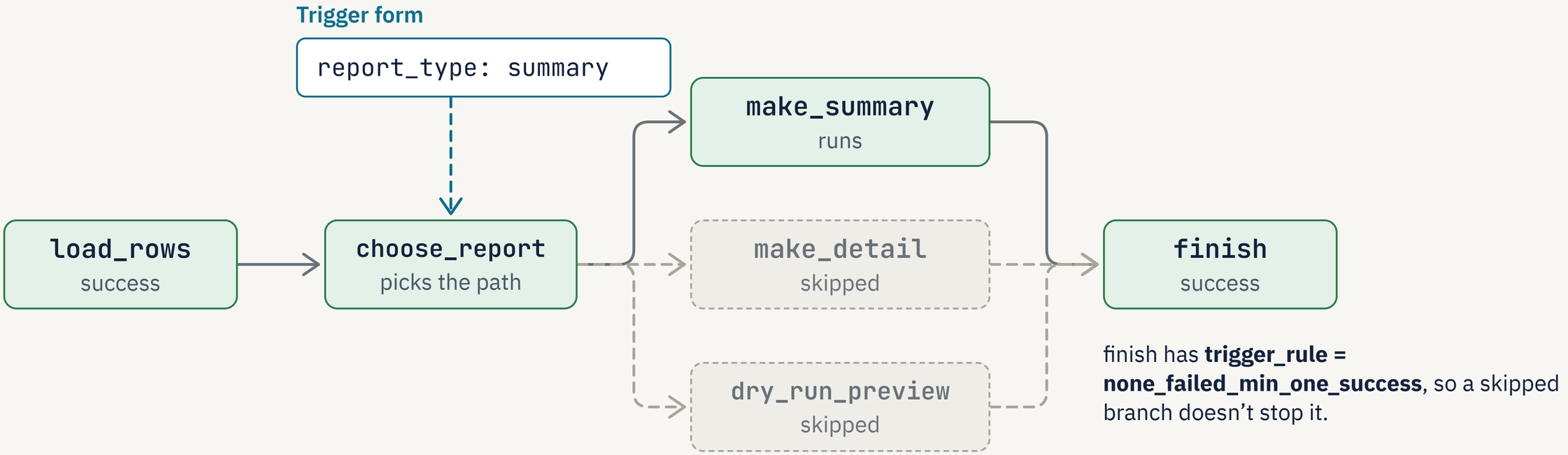
THE EXAMPLE JOBS

Seven examples, one new idea each



EXAMPLE 4 · BRANCHING

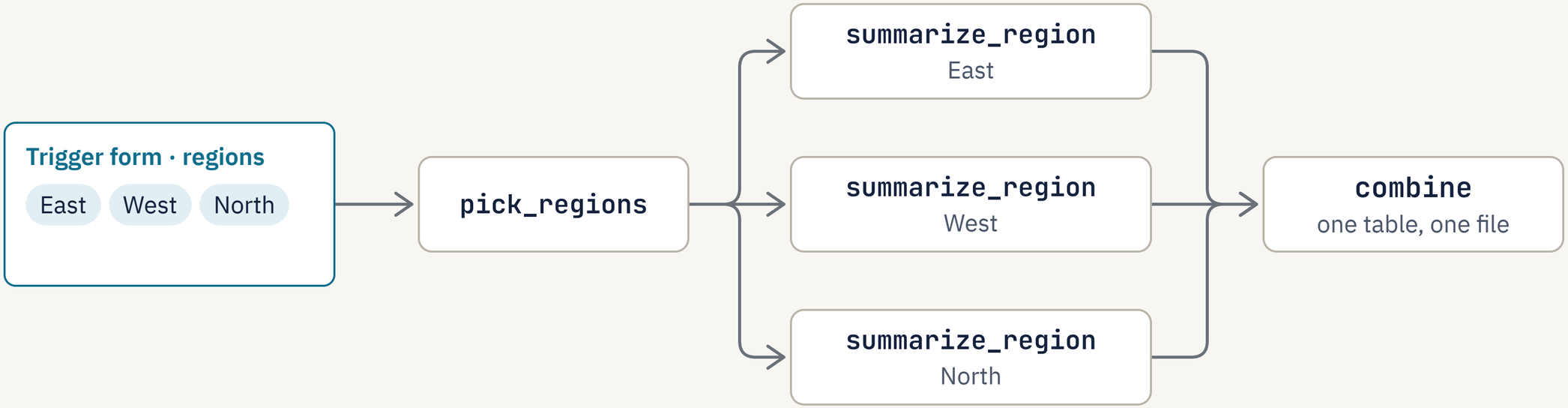
An input decides which steps run



Set report_type to **both** and both make_ steps run; tick **dry_run** and only the preview runs.

EXAMPLE 5 · ONE STEP PER ITEM

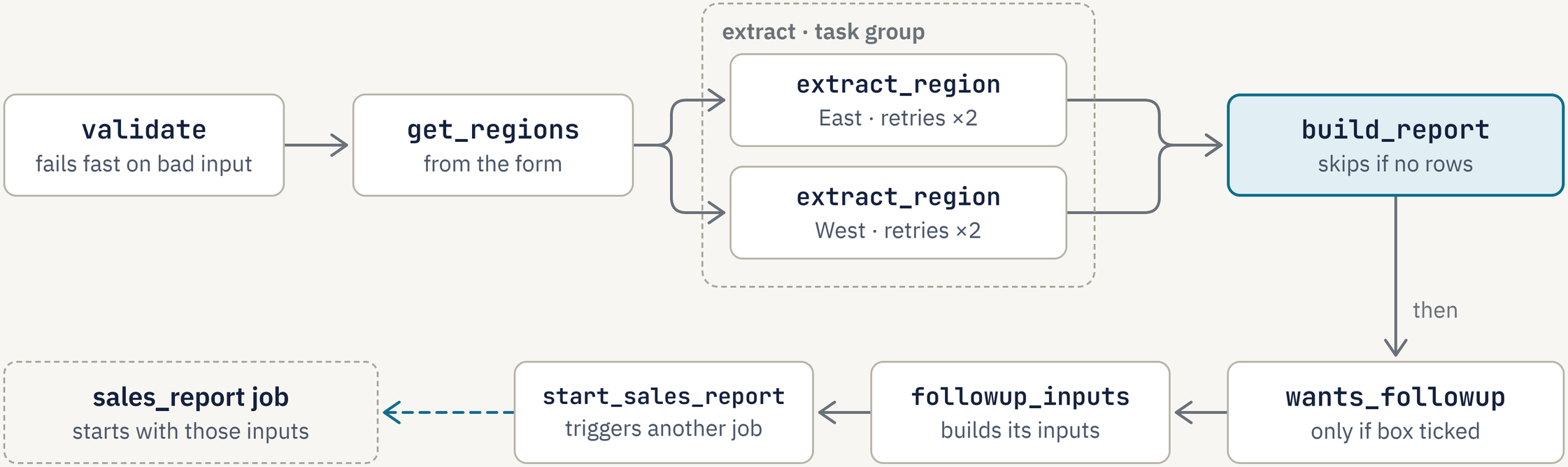
Pick 3 regions, get 3 parallel steps



.expand(region=...) makes one copy of the step per item picked. Each copy has its own log and can be rerun alone.

EXAMPLE 7 · EVERYTHING TOGETHER

A production-style manual job



Every step retries, then logs to failures.log · one run at a time · manifest.json records who ran it, with which inputs

DEPENDENCIES IN CODE

>> draws the arrows between tasks

```
a >> b >> c
```

A chain: one after another

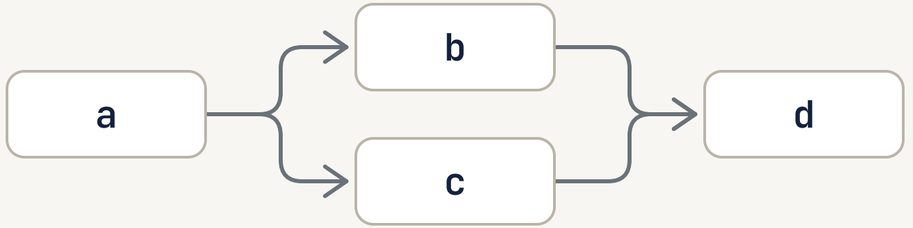
```
a >> [b, c] >> d
```

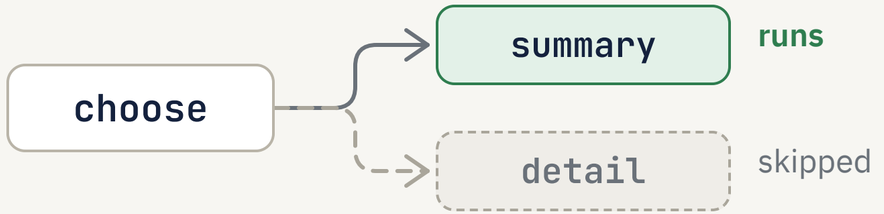
Fan out, then fan in

```
choose >> [summary, detail]
```

A branch picks one path

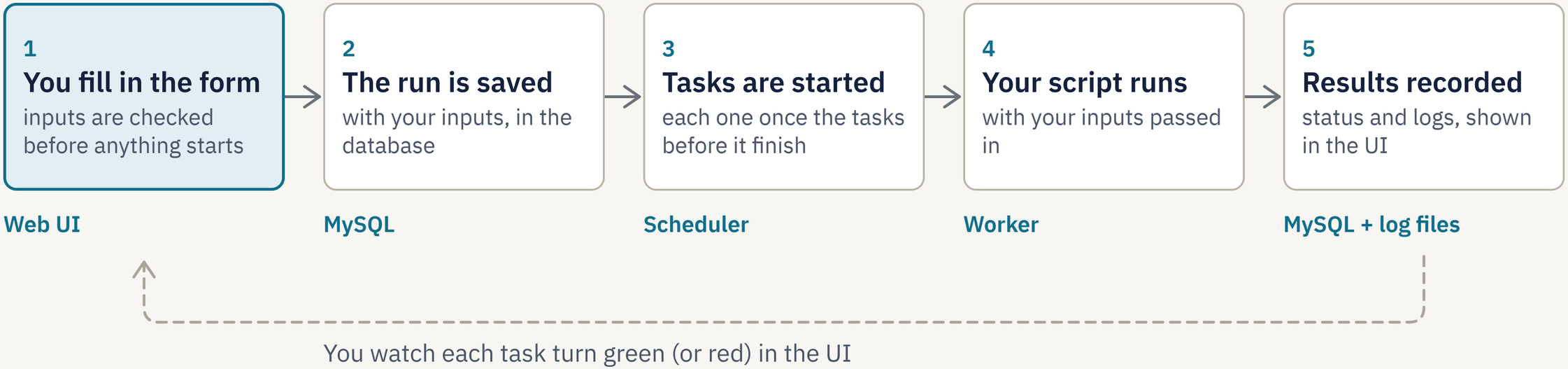






BEHIND THE TRIGGER BUTTON

What happens after you click Trigger



The same path is used when the run is started from the command line or by another job.

VOCABULARY

Five terms cover almost everything

Term	Means	In our demo
DAG	A job: steps and their order, one Python file	sales_report
Task	One step in the job	filter_data, summarize
Params	The job’s inputs, shown as a form	region, min_amount
DAG run	One execution, with its own inputs and logs	Each click of Trigger
Scheduler	The background service that starts tasks	Runs on the server

WHERE FILES GO

Only job files go in the dags folder; scripts live anywhere

```
$AIRFLOW_HOME/dags/
  03_sales_report.py

/opt/jobs/
  scripts/filter_data.py
  scripts/summarize.py
  data/sales.csv
```

dags folder: Airflow scans it for job definitions every few minutes.

Scripts and data: any folder on the server. Call them by full path.

Relative paths break, because tasks don’t run from the DAG file’s folder.

CHEAT SHEET

Six commands worth remembering

Command	Does
`airflow dags list`	List all jobs Airflow found
`airflow dags list-import-errors`	Show job files that failed to load
`airflow dags trigger <job> --conf '{…}'`	Start a run with inputs
`airflow dags test <job>`	Run once in the terminal
`airflow tasks test <job> <task>`	Run a single step
`airflow version`	Show the installed version

FURTHER READING

References

Resource	Use it for
This session's code and slides github.com/lllatsyrC/airflow-demo	The 7 example jobs, sample data, setup steps
Airflow docs: Architecture Overview airflow.apache.org/docs	What each server part does (Airflow 3)
Airflow docs: Params core-concepts/params	Every trigger-form input type
shyarnis/airflow-presentation github.com/shyarnis	Teaching slides on DAGs; install steps are for Airflow 2.7
Data Pipelines with Apache Airflow Harenslak & de Ruiter, Manning	The book behind the "manual steps to tasks" and "no loops" ideas